第 4 章 栈和 stack

1. 栈的概念

（可以用 ppt 演示一下栈的相关概念）

栈是一种只允许在一端进行数据插入和删除操作的**线性表**。

- 进行数据插入或删除的一端称为**栈顶**，另一端称为**栈底**。不含元素的栈称为**空栈**。
- **进栈**就是往栈中放入元素，**出栈**就是将元素弹出栈顶。

栈其实是一个比较简单的数据结构。学习的重点在于用栈去解决问题，这也是难点。

【注意】

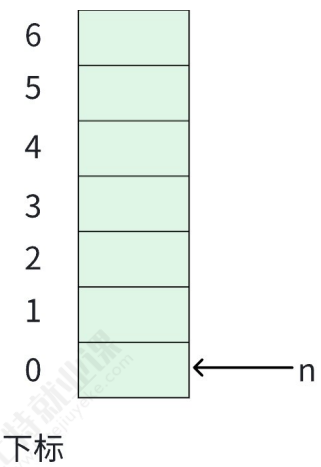
> 如果定义了一个栈结构，那么添加和删除元素只能在栈顶进行。不能随意位置添加和删除元素，这是栈这个数据结构的特性，也是规定。

2. 栈的模拟实现

（比较简单，就不用 ppt 展示了，用课件上的图足矣）

2.1 创建

1. 本质还是线性表，因此可以创建一个足够大的数组，充当栈结构
2. 再定义一个变量 `n`，用来记录栈中元素的个数，同时还可以标记栈顶的位置。



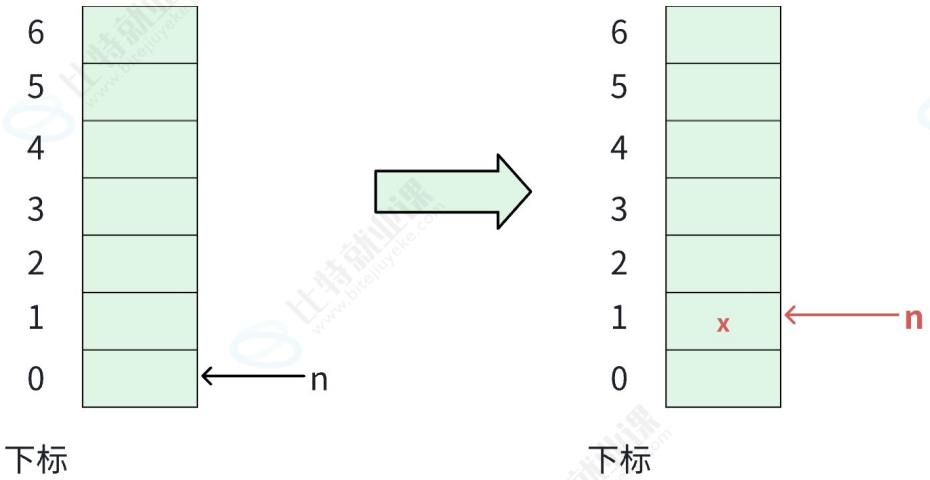
代码实现：

```cpp
const int N = 1e6 + 10;

int n;
int stk[N];
```

2.2 进栈

> 这里依旧舍弃下标为 0 的位置，有效元素从 1 开始记录。
>
> 进栈操作，那就把元素放在栈顶位置即可。



代码实现：

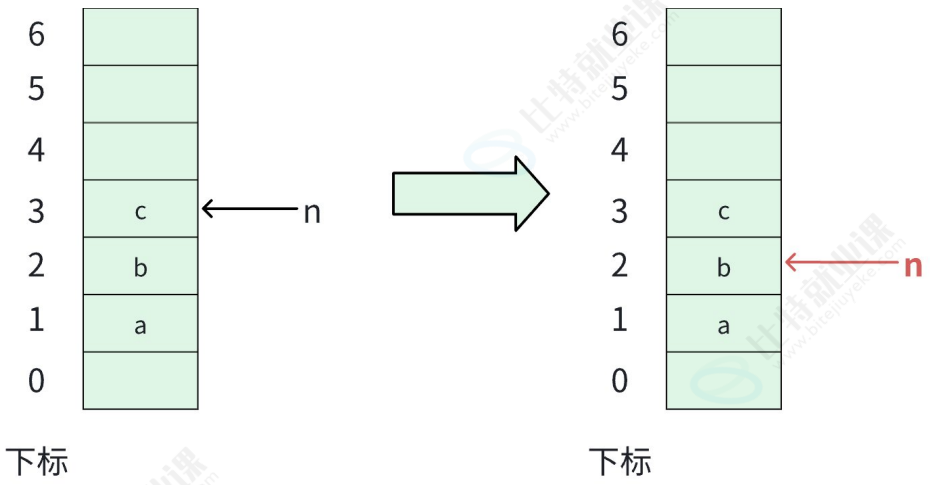
```cpp
// 进栈
void push(int x)
{
    stk[++n] = x;
}
```

时间复杂度：

> 显然是 $O(1)$。

2.3 出栈

> 不用真的删除元素，只用将元素个数减 1 ，就相当于删除栈顶元素。



代码实现：

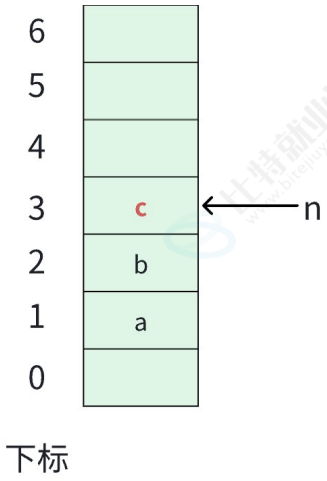
```
1   // 出栈
2   void pop()
3   {
4       n--;
5   }
```

时间复杂度：

> 显然是 $O(1)$ 。

2.4 栈顶元素

> 查询栈顶元素。
>
> 这里要注意，因为栈特殊的规定，不支持遍历整个栈中的元素。因此，需要查找栈中元素的时候，只能查找到栈顶元素。



代码实现：

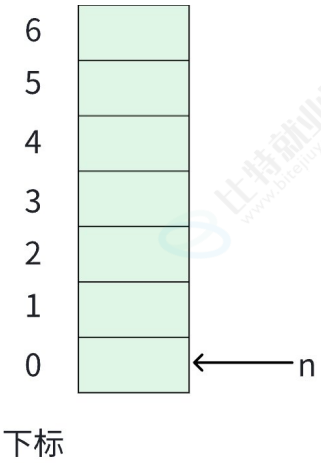
```
1  // 栈顶元素
2  int top()
3  {
4      return stk[n];
5  }
```

时间复杂度：

> 显然是 $O(1)$。

2.5 判空

> 判断栈是否为空



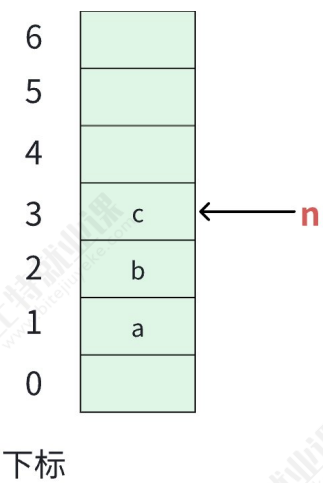
代码实现：

```
1   // 判空
2   bool empty()
3   {
4       return n == 0;
5   }
```

时间复杂度：

> 显然是 $O(1)$。

2.6 有效元素的个数



代码实现：

```
// 栈中元素个数
int size()
{
    return n;
}
```

时间复杂度：

> 显然是 $O(1)$。

2.7 所有测试代码

```
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

// 创建栈
int stk[N], n;

// 进栈 - 本质就是顺序表里面的尾插
void push(int x)
{
    stk[++n] = x;
}

// 出栈 - 顺序表的尾删操作
void pop()
{
    n--;
}

// 查询栈顶元素
int top()
{
    return stk[n];
}
```

```
27
28  // 判断是否为空
29  bool empty()
30  {
31      return n == 0;
32  }
33
34  // 查询有效元素的个数
35  int size()
36  {
37      return n;
38  }
39
40  int main()
41  {
42      for(int i = 1; i <= 10; i++)
43      {
44          push(i);
45      }
46
47      // 当栈不为空的时候
48      while(size()) // while(!empty())
49      {
50          cout << top() << endl;
51          pop();
52      }
53
54
55      return 0;
56  }
```

3. stack

> 有了之前 `vector` 和 `list` 的铺垫，栈的使用应该会比较得心应手。
>
> 1. 如何创建?
> 2. 关心里面有什么函数?
> 3. 函数的功能以及时间复杂度

3.1 创建

> `stack<T> st;`
>
> `T` 可以是任意类型的数据。

3.2 size / empty

1. `size`：返回栈里实际元素的个数；
2. `empty`：返回栈是否为空。

时间复杂度：$O(1)$。

3.3 push / pop

1. `push`：进栈；
2. `pop`：出栈。

时间复杂度：$O(1)$。

3.4 top

1. `top`：返回栈顶元素，但是不会删除栈顶元素。

时间复杂度：$O(1)$。

3.5 所有测试代码：

```cpp
#include <iostream>
#include <stack>

using namespace std;

int main()
{
    stack<int> st;

    // 先讲 1~10 进栈
    for(int i = 1; i <= 10; i++)
    {
        st.push(i);
    }

    while(st.size()) // !st.empty()
    {
        cout << st.top() << endl;
```

```
19            st.pop();
20        }
21
22        return 0;
23    }
```

4. 算法题

4.1 【模板】栈

> **题目来源：** 洛谷
>
> **题目链接：** B3614 【模板】栈
>
> **难度系数：** ★

【题目描述】

> 请你实现一个栈（stack），支持如下操作：
>
> - `push(x)`：向栈中加入一个数 x。
> - `pop()`：将栈顶弹出。如果此时栈为空则不进行弹出操作，输出 `Empty`。
> - `query()`：输出栈顶元素，如果此时栈为空则输出 `Anguei!`。
> - `size()`：输出此时栈内元素个数。

【输入描述】

> 本题单测试点内有多组数据。
>
> 输入第一行是一个整数 T，表示数据组数。对于每组数据，格式如下：
>
> - 每组数据第一行是一个整数，表示操作的次数 n。
> - 接下来 n 行，每行首先由一个字符串，为 `push`，`pop`，`query` 和 `size` 之一。若为 `push`，则其后有一个整数 x，表示要被加入的数，x 和字符串之间用空格隔开；若不是 `push`，则本行没有其它内容。

【输出描述】

> 对于每组数据，按照「题目描述」中的要求依次输出。每次输出占一行。

【示例一】

> 输入：
>
> ```
> 2
> 5
> ```

> push 2
> query
> size
> pop
> query
> 3
> pop
> query
> size
> 输出:
> 2
> 1
> Anguei!
> Empty
> Anguei!
> 0

【解法】

模拟出一个栈结构即可。

【参考代码】

```cpp
#include <iostream>

using namespace std;

typedef unsigned long long LL;
const int N = 1e6 + 10;

int top;
LL stk[N];

int main()
{
    int T; cin >> T;
    while(T--)
    {
```

```
        // 清空栈
        top = 0;

        int n; cin >> n;
        while(n--)
        {
            string op; cin >> op;
            if(op == "push") // 入栈
            {
                LL x; cin >> x;
                stk[++top] = x;
            }
            else if(op == "pop") // 出栈
            {
                if(top == 0) cout << "Empty" << endl;
                else top--;
            }
            else if(op == "query") // 查询
            {
                if(top == 0) cout << "Anguei!" << endl;
                else cout << stk[top] << endl;
            }
            else // 栈的大小
            {
                cout << top << endl;
            }
        }
    }

    return 0;
}
```

4.2 有效的括号

> **题目来源：** 力扣
>
> **题目链接：** 20. 有效的括号
>
> **难度系数：** ★

【题目描述】

> 给定一个只包括 `'('`，`')'`，`'{'`，`'}'`，`'['`，`']'` 的字符串 `s` ，判断字符串是否有效。
>
> 有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

【示例一】

> **输入：** s = "()"
>
> **输出：** true

【示例二】

> **输入：** s = "()[]{}"
>
> **输出：** true

【解法】

用栈来模拟括号匹配的过程：

- 如果是左括号，就进栈；
- 如果是右括号，就看看栈里面有没有左括号与之匹配：
  - 如果有，继续模拟；
  - 如果没有，说明不是合法的括号序列

【参考代码】

```
class Solution
{
public:
    bool isValid(string s)
    {
        stack<char> st;

        for(auto ch : s)
        {
            if(ch == '[' || ch == '(' || ch == '{')
            {
                st.push(ch);
            }
            else
            {
                if(st.empty()) return false;
                char left = st.top();
                if(ch == ')' && left != '(') return false;
                if(ch == ']' && left != '[') return false;
```

```
                if(ch == '}' && left != '{') return false;

                st.pop();
            }
        }

        return st.empty();
    }
};
```

4.3 验证栈序列

题目来源： 洛谷

题目链接： P4387【深基15.习9】验证栈序列

难度系数： ★

【题目描述】

给出两个序列 `pushed` 和 `poped` 两个序列，其取值从 1 到 $n(n \leq 100000)$。

已知入栈序列是 `pushed`，如果出栈序列有可能是 `poped`，则输出 `Yes`，否则输出 `No`。为了防止骗分，每个测试点有多组数据，不超过 5 组。

【输入描述】

第一行一个整数 q，询问次数。

接下来 q 个询问，对于每个询问：

第一行一个整数 n 表示序列长度；

第二行 n 个整数表示入栈序列；

第三行 n 个整数表示出栈序列；

【输出描述】

对于每个询问输出答案。

【示例一】

输入：

```
2
5
1 2 3 4 5
5 4 3 2 1
```

> 4
>
> 1 2 3 4
>
> 2 4 1 3
>
> 输出：
>
> Yes
>
> No

【解法】

用栈来模拟进出栈的流程。

一直让元素进栈，进栈的同时判断是否需要出栈。当所有元素模拟完毕之后，如果栈中还有元素，那么就是一个非法的序列。否则，就是一个合法的序列。

【参考代码】

```
#include <iostream>
#include <stack>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N], b[N];

int main()
{
    int q; cin >> q;
    while(q--)
    {
        cin >> n;
        for(int i = 1; i <= n; i++) cin >> a[i];
        for(int i = 1; i <= n; i++) cin >> b[i];

        stack<int> st;
        int j = 1; // 标记当前出栈的元素
        for(int i = 1; i <= n; i++)
        {
            // 进栈
            st.push(a[i]);
            // 判断是否出栈
            while(j <= n && st.size() && b[j] == st.top())
```

```
            {
                st.pop();
                j++;
            }
        }
        if(st.empty()) cout << "Yes" << endl;
        else cout << "No" << endl;
    }


    return 0;
}
```

4.4 后缀表达式

题目来源： 洛谷

题目链接： P1449 后缀表达式

难度系数： ★

【题目描述】

所谓后缀表达式是指这样的一个表达式：式中不再引用括号，运算符号放在两个运算对象之后，所有计算按运算符号出现的顺序，严格地由左而右新进行（不用考虑运算符的优先级）。

本题中运算符仅包含 `+-*/+-*/` 。保证对于 `/` 运算除数不为 0 。特别地，其中 `/` 运算的结果需要向 0 取整（即与 C++ `/` 运算的规则一致）。

如： `3*(5-2)+7` 对应的后缀表达式为： `3.5.2.-*7.+@` 。在该式中， `@` 为表达式的结束符号。 `.` 为操作数的结束符号。

【输入描述】

输入一行一个字符串 s ，表示后缀表达式。

数据保证， $1 \leq |s| \leq 50$ ，答案和计算过程中的每一个值的绝对值不超过 10^9 。

【输出描述】

输出一个整数，表示表达式的值。

【示例一】

输入：

```
3.5.2.-*7.+@
```

输出：

```
16
```

【示例二】

> 输入：
>
> 10.28.30./*7.-@
>
> 输出：
>
> -7

【解法】

用栈模拟后缀表达式的计算过程。

【参考代码】

```cpp
#include <iostream>
#include <stack>

using namespace std;

int main()
{
    stack<int> st;

    int num = 0;
    char ch;

    while(cin >> ch)
    {
        if(ch == '@') break;
        if(ch == '.')
        {
            st.push(num);
            num = 0;
        }
        else if(ch <= '9' && ch >= '0')
        {
            num = num * 10 + ch - '0';
        }
        else
        {
            // 操作符
            int b = st.top(); st.pop(); // 先出栈的是右操作数
            int a = st.top(); st.pop();
            if(ch == '+') st.push(a + b);
            else if(ch == '-') st.push(a - b);
            else if(ch == '*') st.push(a * b);
```

```
33                 else st.push(a / b);
34             }
35         }
36         cout << st.top() << endl;
37 
38         return 0;
39     }
```

4.5 括号序列

题目来源： 洛谷

题目链接： P1241 括号序列

难度系数： ★

【题目描述】

定义如下规则：

1. 空串是「平衡括号序列」
2. 若字符串 S 是「平衡括号序列」，那么 $[S]$ 和 (S) 也都是「平衡括号序列」
3. 若字符串 A 和 B 都是「平衡括号序列」，那么 AB（两字符串拼接起来）也是「平衡括号序列」。

例如，下面的字符串都是平衡括号序列：

- `()`，`[]`，`(())`，`([])`，`()[]`，`()[()]`

而以下几个则不是：

- `(`，`[`，`]`，`)(`，`())`，`([()`

现在，给定一个仅由 `(`，`)`，`[`，`]` 构成的字符串 s，请你按照如下的方式给字符串中每个字符配对：

1. 从左到右扫描整个字符串。
2. 对于当前的字符，如果它是一个右括号，考察它与它左侧离它最近的未匹配的的左括号。如果该括号与之对应（即小括号匹配小括号，中括号匹配中括号），则将二者配对。如果左侧未匹配的左括号不存在或与之不对应，则其配对失败。

配对结束后，对于 s 中全部未配对的括号，请你在其旁边添加一个字符，使得该括号和新加的括号匹配。

【输入描述】

输入只有一行一个字符串，表示 s。

> 对于全部的测试点，保证 s 的长度不超过 100，且只含 `(`，`)`，`[`，`]` 四种字符。

【输出描述】

> 输出一行一个字符串表示你的答案。

【示例一】

> 输入：
>
> ([()
>
> 输出：
>
> ()[]()

【示例二】

> 输入：
>
> ([)
>
> 输出：
>
> ()[]()

【解法】

先用栈模拟一遍括号匹配的过程，同时用一个 $bool$ 类型的数组标记哪些位置可以匹配，哪些位置不可以匹配。

模拟结束之后，把未匹配位置的左括号，添上一个相应的右括号。

【参考代码】

```
#include <iostream>
#include <stack>

using namespace std;

const int N = 110;

bool st[N]; // 标记有没有匹配

int main()
{
    string s; cin >> s;
    int n = s.size();
    stack<int> stk; // 里面存的是左括号的下标
    string ret;

    for(int i = 0; i < n; i++)
```

```
18      {
19          if(s[i] == '(' || s[i] == '[')
20          {
21              stk.push(i);
22          }
23          else
24          {
25              if(stk.empty()) continue;
26
27              char ch = s[stk.top()];
28              if((ch == '(' && s[i] == ')') || (ch == '[' && s[i] == ']')) // 能
    匹配
29              {
30                  st[stk.top()] = true;
31                  st[i] = true;
32                  stk.pop();
33              }
34          }
35      }
36
37      for(int i = 0; i < n; i++)
38      {
39          char ch = s[i];
40          if(st[i]) ret += ch;
41          else
42          {
43              // 不能匹配就要加括号
44              if(ch == '(')
45              {
46                  ret += ch;
47                  ret += ')';
48              }
49              else if(ch == ')')
50              {
51                  ret += '(';
52                  ret += ch;
53              }
54              else if(ch == '[')
55              {
56                  ret += ch;
57                  ret += ']';
58              }
59              else if(ch == ']')
60              {
61                  ret += '[';
62                  ret += ch;
63              }
```

```
64            }
65        }
66
67        cout << ret << endl;
68
69        return 0;
70    }
```